

Name: Mimanshu Gahlaut

UID: 24BAI70038

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment 9: Database Triggers

1. Aim of the Session

To understand and implement database triggers in PostgreSQL to automate data validation and computational logic, ensuring data integrity by enforcing business rules during DML operations.

2. Software Requirements:

- Database Management System:
 - Oracle Database Express Edition (Oracle XE)
 - PostgreSQL Database
- Database Administration Tool / Client Tool:
 - Oracle SQL Developer (for Oracle XE)
 - pgAdmin (for PostgreSQL)

3. Objective of the Session

- To create a trigger function that performs automatic calculations on row- level data.
- To define a BEFORE INSERT trigger that intercepts data entry for validation.
- To implement custom exception handling using the RAISE EXCEPTION command.
- To verify trigger behaviour by testing both valid and invalid data scenarios within a transaction.

4. Practical / Experiment Steps

The work was carried out through the following activities:

1. **Table Setup:** Created an employee table consisting of attributes such as working hours, hourly salary, and a computed salary field.
2. **Function Implementation:** Developed a PL/pgSQL trigger function to calculate total salary based on input values.

3. **Validation Condition:** Added a rule within the function to prevent insertion if the calculated salary crosses a specified threshold.
4. **Trigger Creation:** Defined a row-level trigger to execute before each INSERT operation on the table.
5. **Testing Phase:** Executed multiple test cases using anonymous blocks to verify proper trigger execution and validation.

5. Procedure of the Practical

Execution was performed in the following order:

- Opened PostgreSQL environment using pgAdmin.
- Created the employee table with necessary fields for computation.
- Defined a trigger function using PL/pgSQL to calculate salary automatically.
- Utilized the NEW keyword to access incoming tuple values before insertion.
- Incorporated a condition to check whether the computed salary exceeds a limit.
- Used RAISE EXCEPTION to block invalid data entries.
- Created a BEFORE INSERT trigger linked to the table for row-wise execution.
- Inserted valid data to confirm successful computation and storage.
- Attempted insertion of invalid data to verify exception handling.
- Reviewed table contents to ensure only valid records were stored.

6. I/O Analysis (Input / Output Analysis)

Input Queries

SQL

```
CREATE TABLE employee2 (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    working_hours INT,  
    perhour_salary NUMERIC,  
    total_payable_amount NUMERIC  
);
```

Query	Query History
1	CREATE TABLE employee2 (2 emp_id INT PRIMARY KEY , 3 emp_name VARCHAR(50) , 4 working_hours INT , 5 perhour_salary NUMERIC , 6 total_payable_amount NUMERIC 7);

Data Output	Messages	Notifications
CREATE TABLE		
Query returned successfully in 137 msec.		

```
CREATE OR REPLACE FUNCTION calculate_amount()  
RETURNS TRIGGER  
AS  
$$  
BEGIN  
    NEW.total_payable_amount := NEW.working_hours * NEW.perhour_salary;  
  
    IF NEW.total_payable_amount > 28000 THEN  
        RAISE EXCEPTION 'Calculated amount exceeds limit';  
    END IF;  
  
    RETURN NEW;  
END;  
$$  
LANGUAGE PLPGSQL;
```

10	CREATE OR REPLACE FUNCTION calculate_amount() 11 RETURNS TRIGGER 12 AS 13 \$\$ 14 BEGIN 15 NEW.total_payable_amount := NEW.working_hours * NEW.perhour_salary; 16 17 IF NEW.total_payable_amount > 28000 THEN 18 RAISE EXCEPTION 'Calculated amount exceeds limit'; 19 END IF ; 20 21 RETURN NEW ; 22 END ; 23 \$\$ 24 LANGUAGE PLPGSQL ;
----	--

Data Output	Messages	Notifications
CREATE FUNCTION		
Query returned successfully in 136 msec.		

```

CREATE OR REPLACE TRIGGER payable_trigger
BEFORE INSERT
ON employee2
FOR EACH ROW
EXECUTE FUNCTION calculate_amount();

```

```

26
27 CREATE OR REPLACE TRIGGER payable_trigger
28 BEFORE INSERT
29 ON employee
30 FOR EACH ROW
31 EXECUTE FUNCTION calculate_amount();
32
33
34

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 125 msec.

```

DO
$$
BEGIN
    INSERT INTO employee2(emp_id, emp_name, working_hours, perhour_salary)
    VALUES (1, 'Rohit', 8, 300);

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE '%', SQLERRM;
END;
$$;

SELECT * FROM employee2;

```

```

35 DO
36 $$
37 BEGIN
38     INSERT INTO employee2(emp_id, emp_name, working_hours, perhour_salary)
39     VALUES (1, 'Rohit', 8, 300);
40
41 EXCEPTION
42     WHEN OTHERS THEN
43         RAISE NOTICE '%', SQLERRM;
44 END;
45 $$;
46
47 SELECT * FROM employee2;
48
49

```

Data Output Messages Notifications

SQL

	emp_id [PK] integer	emp_name character varying (50)	working_hours integer	perhour_salary numeric	total_payable_amount numeric
1	1	Rohit	8	300	[null]

```

DO
$$
BEGIN
    INSERT INTO employee2(emp_id, emp_name, working_hours, perhour_salary)
    VALUES (2, 'Aman', 50, 800);

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE '%', SQLERRM;
END;
$$;

```

```

49
50 DO
51 $$
52 BEGIN
53     INSERT INTO employee2(emp_id, emp_name, working_hours, perhour_salary)
54     VALUES (2, 'Aman', 50, 800);
55
56 EXCEPTION
57     WHEN OTHERS THEN
58         RAISE NOTICE '%', SQLERRM;
59 END;
60 $$;
61 |
62 SELECT * FROM employee2;
63
Data Output  Messages  Notifications
NOTICE:  Calculated amount exceeds limit
DO
Query returned successfully in 160 msec.

```

```
SELECT * FROM employee2;
```

```

61
62 SELECT * FROM employee2;
63
Data Output  Messages  Notifications

```

	emp_id [PK] integer	emp_name character varying (50)	working_hours integer	perhour_salary numeric	total_payable_amount numeric
1	1	Rohit	8	300	2400

7. Learning Outcome

- **Understanding Trigger Execution:** Learned how triggers automatically execute in response to database events.
- **Dynamic Data Handling:** Gained experience in computing values during insertion without manual input.
- **Validation Mechanism:** Understood how triggers enforce business rules effectively.
- **Exception Management:** Learned how to handle errors using RAISE EXCEPTION.
- **Data Integrity Assurance:** Recognized the importance of triggers in maintaining consistency and correctness of data.